

API 说明

当前接口已按真实后端接入方式封装，默认走 Mock。由于后端字段结构还未最终确定，前端增加了 `api/adapters` 适配层：页面只消费稳定的前端领域模型，后端返回结构变化时优先修改 adapter，不直接改页面。

通用响应

```
interface ApiEnvelope<T> {  
  code: number;  
  message: string;  
  data: T;  
}
```

成功示例：

```
{  
  "code": 0,  
  "message": "ok",  
  "data": {}  
}
```

错误示例：

```
{  
  "code": 40001,  
  "message": "文件格式不支持"  
}
```

Axios 错误会在 `src/api/client.ts` 中统一转换为 `Error(message)`。

请求拦截器会自动添加：

```
X-Request-Id: <uuid>
```

后端可将该值写入日志，方便联调追踪。

FastAPI 校验错误

FastAPI 常见 422 返回：

```
{
  "detail": [
    {
      "loc": ["body", "file"],
      "msg": "Field required",
      "type": "missing"
    }
  ]
}
```

前端会格式化为：

```
body.file: Field required
```

如果后端返回 `ApiEnvelope<T>` 且 `code !== 0`，`unwrapApiEnvelope` 会抛出业务错误，页面通过统一错误状态展示。

数据适配层

核心文件：

```
src/api/adapters/common.ts
src/api/adapters/algorithmAdapter.ts
```

适配层当前支持：

- `ApiEnvelope<T>`: { code, message, data }
- 裸数据：直接返回算法结果对象
- 常见替代包装：{ data }、{ result }
- `snake_case` 和 `camelCase` 字段
- 常见数组路径差异，例如 `multiparameter`、`multi_parameter`、`records.multiparameter`、`algorithm_results.multiparameter`
- 字段缺失时使用安全默认值，保证页面可展示

页面、图表、报告不直接依赖后端 DTO。后端结构变化时，优先修改：

```
normalizeAlgorithmDataset(payload)
normalizePatientInfo(payload)
```

获取算法结果

文件： `src/api/algorithm.ts`

```
GET /algorithm/results
```

返回:

```
interface AlgorithmDataset {  
  patient: PatientInfo;  
  detection: DetectionInfo;  
  multiparameter: MultiParameterRecord[];  
  sleep: SleepRecord[];  
  sleepEvents: SleepEvent[];  
  emg: EmgRecord[];  
}
```

后端实际返回可以是下面任意一种，前端都会尝试适配:

```
{  
  "code": 0,  
  "message": "ok",  
  "data": {  
    "patient_info": {},  
    "detection_info": {},  
    "multi_parameter": [],  
    "sleep_records": [],  
    "sleep_events": [],  
    "emg_records": []  
  }  
}
```

```
{  
  "patient": {},  
  "detection": {},  
  "records": {  
    "multiparameter": [],  
    "sleep": [],  
    "emg": []  
  }  
}
```

Mock 来源:

```
src/mock/algorithm.ts
```

获取患者信息

文件: `src/api/patient.ts`

```
GET /patients/{patientId}
```

返回:

```
interface PatientInfo {  
  patientId: string;  
  name: string;  
  gender: '男' | '女';  
  age: number;  
  heightCm: number;  
  weightKg: number;  
  department: string;  
  bedNo: string;  
}
```

上传数据文件

文件: `src/api/upload.ts`

```
POST /uploads  
Content-Type: multipart/form-data
```

请求字段:

```
file: JSON 或 CSV 文件
```

返回:

```
interface UploadParseResult {  
  meta: UploadFileMeta;  
  records: ImportedPreviewRecord[];  
}
```

Mock 模式下不会发请求, 而是在浏览器本地执行:

```
parseDataFile(file);
```

支持 JSON:

```
[
  {
    "patient_id": "P001",
    "heart_rate": 72,
    "spo2": 98
  }
]
```

支持 CSV:

```
patient_id,heart_rate,spo2
P001,72,98
P002,75,97
```

生成报告

文件: `src/api/report.ts`

```
POST /reports/generate
```

返回:

```
interface ReportPayload {
  patientId: string;
  detectionId: string;
  generatedAt: string;
  conclusion: string;
  advice: string[];
}
```

当前报告页面使用前端 Mock 文案渲染,未来可以把该接口返回结果接入 `ReportDocument`。

接真实接口步骤

1. 后端可以先返回现有结构,不必一次性完全对齐前端模型。
2. 设置 `VITE_USE MOCK=false`。
3. 设置 `VITE_API_BASE_URL`。
4. 如字段名不同,在 `src/api/adapters/` 增加 DTO 转换。
5. 保持页面仍只消费 `types/` 中的业务类型。

推荐 FastAPI 形态

```
from pydantic import BaseModel

class ApiEnvelope[T](BaseModel):
    code: int = 0
    message: str = "ok"
    data: T
```

上传接口建议使用标准 multipart:

```
@router.post("/uploads")
async def upload(file: UploadFile):
    ...
```

算法结果接口建议一次返回页面所需的核心数据，避免 Demo 阶段多接口瀑布请求。字段名暂不稳定时，由前端 adapter 做兼容。